

PROJECT REPORT

HOMEWORK 4

Written and Prepared by:

Roxanne Lutz

University of New Mexico

October 17, 2025

HANDWORKED PROBLEMS

See following pages for the handwritten, worked problems:

Problem 1) Interp polynomial of degree 20 ($P_{20}(x)$) approximates $f(x) = e^{-x}$ on $[0, 2]$.

Accuracy? Let $n = 20$.

Given the very convenient hint:

$f(x) = e^{-x}$ is a function such that $f^{(20+1)}(x) = f^{(21)}(x)$ is continuous on $[0, 2]$ and $|f^{(21)}(x)| \leq M$, where:

$f^{(21)}(x) \rightarrow 21^{th}$ derivative of e^{-x} .

$\hookrightarrow e^{-x}$ derivative is convenient in that it satisfies:

* Even derivatives: $f^{(2k)}(x) = e^{-x}, k = 0, 1, \dots$

[Odd derivatives: $f^{(2k+1)}(x) = -e^{-x}, k = 0, 1, \dots$

And so, $f^{(21)}(x) = -e^{-x}$

$|f^{(21)}(x)| = e^{-x} \rightarrow$ on $[0, 2]$, Max value of this will be

$x = 0: e^{-(0)} = 1. \quad (\lim_{x \rightarrow \infty} e^{-x} = 0, \text{ thus } x = 0 \text{ is max}).$

So, $M = 1: \boxed{|f^{(21)}(x)| \leq 1.}$

Now, $P_{20}(x)$ is polynomial of degree 20 ($n = 20$) that interpolates $f(x)$ at 21 ($n+1$) equidistant points in $[0, 2]$ including endpoints.

Then by Interpolation Error Theorem II on $[0, 2]$:

$$|f(x) - P_n(x)| \leq \frac{1}{4(n+1)} M \left(\frac{b-a}{n} \right)^{n+1}$$

$$\leq \frac{1}{4(21)} (1) \left(\frac{2-0}{20} \right)^{21} = \frac{1}{84} * (10^{-1})^{21}$$

$$\leq 0.011905 * 10^{-21} = 1.1905 * 10^{-23}$$

And so, the upper bound at error at the $P_{20}(x)$ interpolation

is:

$$\boxed{|f(x) - P_{20}(x)| \leq 1.1905 \times 10^{-23}} \quad (\text{which is highly accurate}).$$

COMPUTER/MATLAB PART 1

ORIGINAL PROBLEM STATEMENT

“Consider the Runge function given by:

$$f(x) = 1/(1 + x^2)$$

Perform the following tasks.”

First, let's (1) code this function to be able to evaluate it more easily, and then (2) plot it to get eyes on it alone.

```
%
=====
(1) RUNGE FUNCTION WRITTEN AS FUNCTION IN runge_function.m FILE
%
=====

function [y] = runge_function(x)
    y = 1 ./ (1 + (x.^ 2));
end

%
=====
(2) GET EXACT VALUES ON [-5, 5] OF RUNGE AND PLOT IN
script_01.m
FILE
%
=====

% preliminary: get f(x) plotted with a fine mesh
start_pt = -5; end_pt = 5;
h = 0.001; % 10,000 points plotted
x_exact = [start_pt:h:end_pt];
y_exact = runge_function(x_exact);
% plot the exact graph only, mainly for ensuring code is correct
figure('Name', 'Exact Runge');
plot( ...
    x_exact, y_exact, "--c", ...
    LineWidth=1.5 ...
```

```

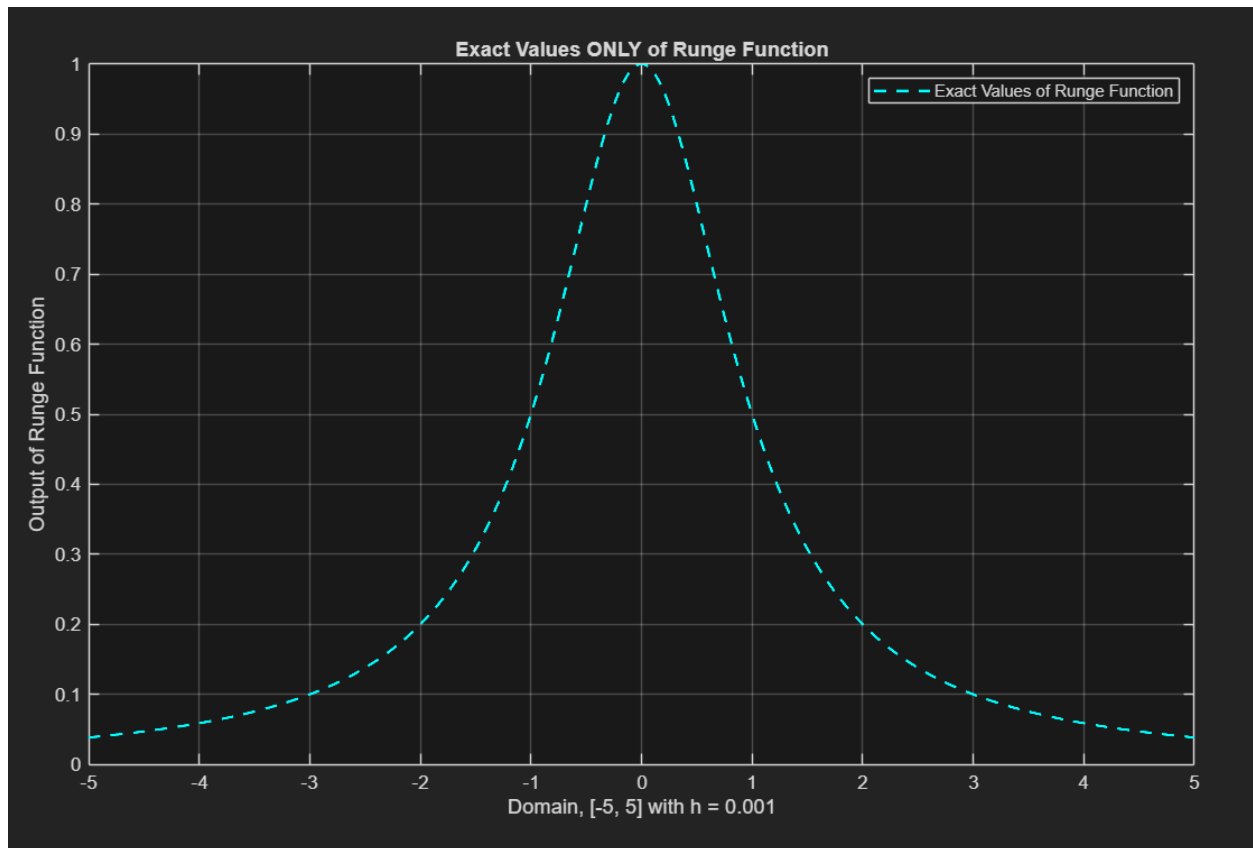
    );
hold on;
legend( ...
    'Exact Values of Runge Function' ...
    );
title('Exact Values ONLY of Runge Function');
xlabel("Domain, [" + start_pt + ", " + end_pt + "] with h = " +
h);
ylabel('Output of Runge Function');
grid on;

%
=====

    END CODE FOR COMP PROB 1
%
=====

```

As follows is a plotting of the exact Runge function by the above script.



As a further preliminary, we are requested to use the proved *Coef* and *Eval* code to find the interpolating polynomial coefficients and then evaluate the interpolating polynomial given the data points and coefficients. The code is pasted below to include for clarity.

```
%
=====

(1) COEF PROVIDED AS FUNCTION IN Coef.m FILE
%
=====

function [a] = Coef(n,x,y)
%Coefficients for Interpolation
clear a;
for i=0:n
    a(i+1)=y(i+1);
end
for j=1:n
    for i=n:-1:j
        a(i+1)=(a(i+1)-a(i+1-1))/(x(i+1)-x(i+1-j));
    end
end
end

%
=====

(2) EVAL PROVIDED AS FUNCTION IN Eval.m FILE
%
=====

function [value] = Eval(n,x,a,t)
%function evaluation - via interpolation
temp=a(end);
for i=n-1:-1:0
    temp=(temp)*(t-x(i+1))+a(i+1);
end
value=temp;
end

%
=====

END CODE FOR COMP PROB 1
```

```
%
```

PART A

“Construct and plot a polynomial of degree 10, that is $P_{10}(x)$ using equally spaced points on the interval $[-5, 5]$.”

Below is the script written to accomplish said task. We will get our equidistance between points by the simple calculation $(end_pt - start_pt) / 2$. Then, we will gather those 11 points, evaluate the exact output of Runge at those points, and finally utilize *Coef* to retrieve the interpolating polynomial coefficients and *Eval* to evaluate the polynomial at a finer mesh of 10,000 points.

```
%
```

```
=====
(1) GENERATION OF THE P10 INTERPOLATION IN script_01.m FILE
=====
```

```
n = 10; % P10, 11 equispaced nodes (n + 1 nodes)
h_equispaced = (end_pt - start_pt) / n; % get equispace
x_P10 = [start_pt:h_equispaced:end_pt];
y_P10 = runge_function(x_P10); % evaluate data points
a_coeff = Coef(n, x_P10, y_P10); % get coefficients

% evaluate the interpolating polynomial at a finer mesh
x_P10_Interp = [start_pt:h:end_pt];
for i = 1:length(x_P10_Interp);
    y_P10_Interp(i) = Eval(n, x_P10, a_coeff, x_P10_Interp(i));
end;

% plot interpolation against exact and data points
figure('Name', 'Interpolation of P10');
plot( ...
    x_exact, y_exact, "--c", ...
    x_P10, y_P10, 'mo', ...
    x_P10_Interp, y_P10_Interp, "--y", ...
    LineWidth=1.5 ...
);
hold on;
legend( ...
```



```

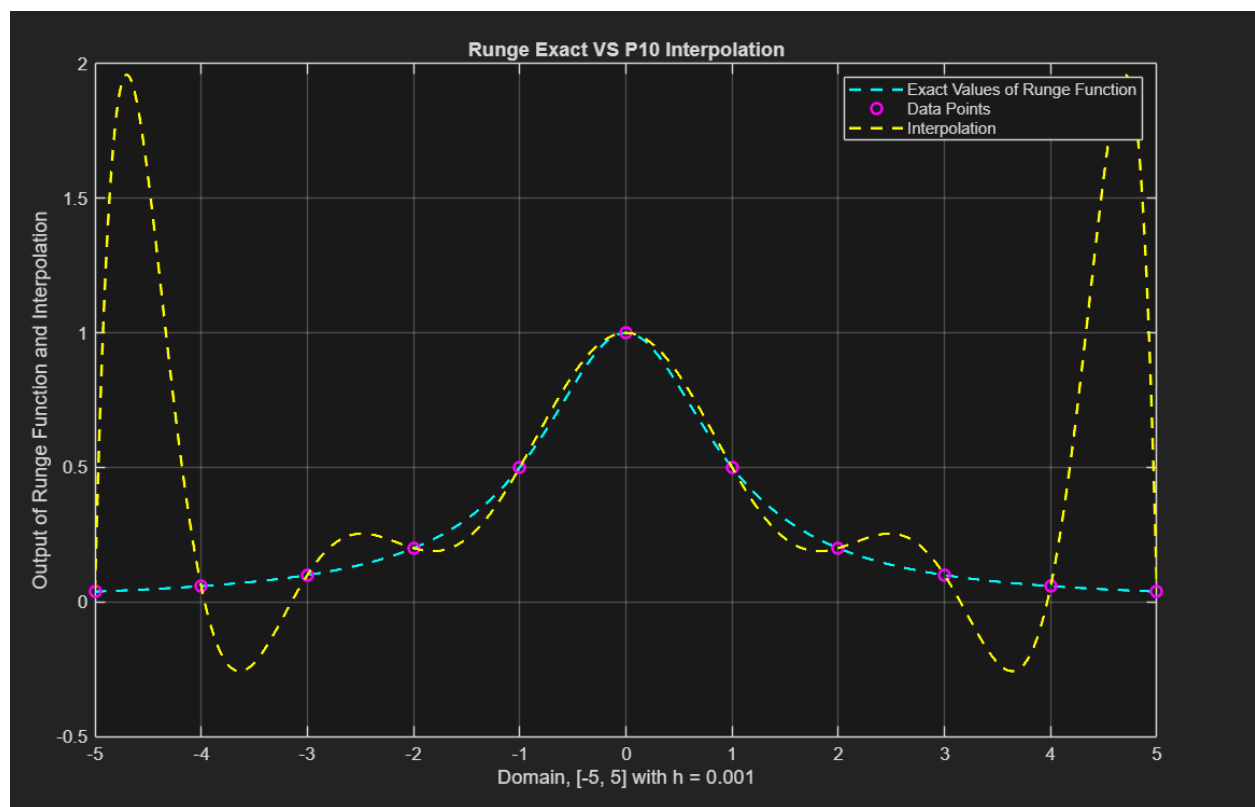
    'Exact Values of Runge Function', ...
    "Data Points", ...
    "Interpolation" ...
);
title('Runge Exact VS P10 Interpolation');
xlabel("Domain, [" + start_pt + ", " + end_pt + "] with h = " +
h);
ylabel('Output of Runge Function and Interpolation');
grid on;

%
=====

    END CODE FOR COMP PROB 1
%
=====

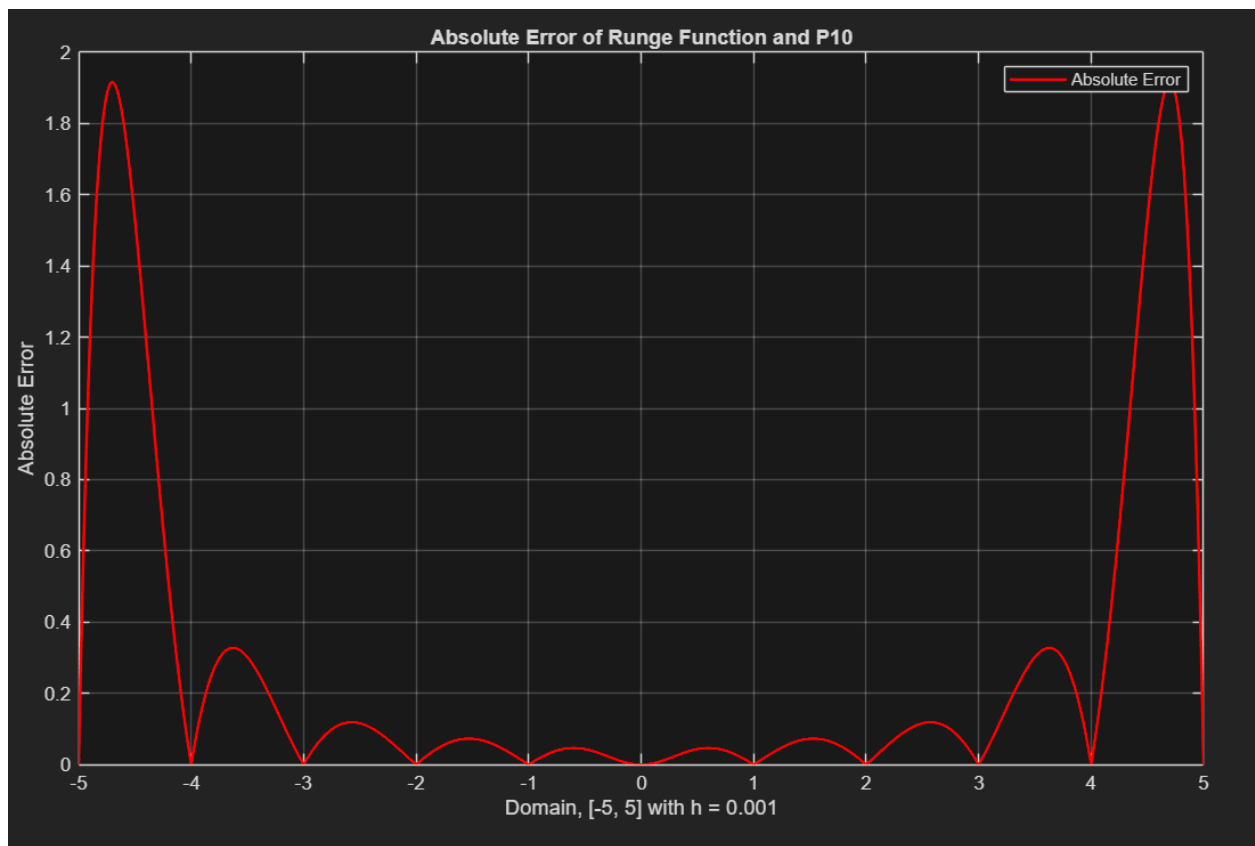
```

The graph generated can be seen below.



Purely visually, we can tell that the equispaced interpolation is fairly accurate near the center, but as we move further in either direction away from $x = 0$, the oscillations of the polynomial

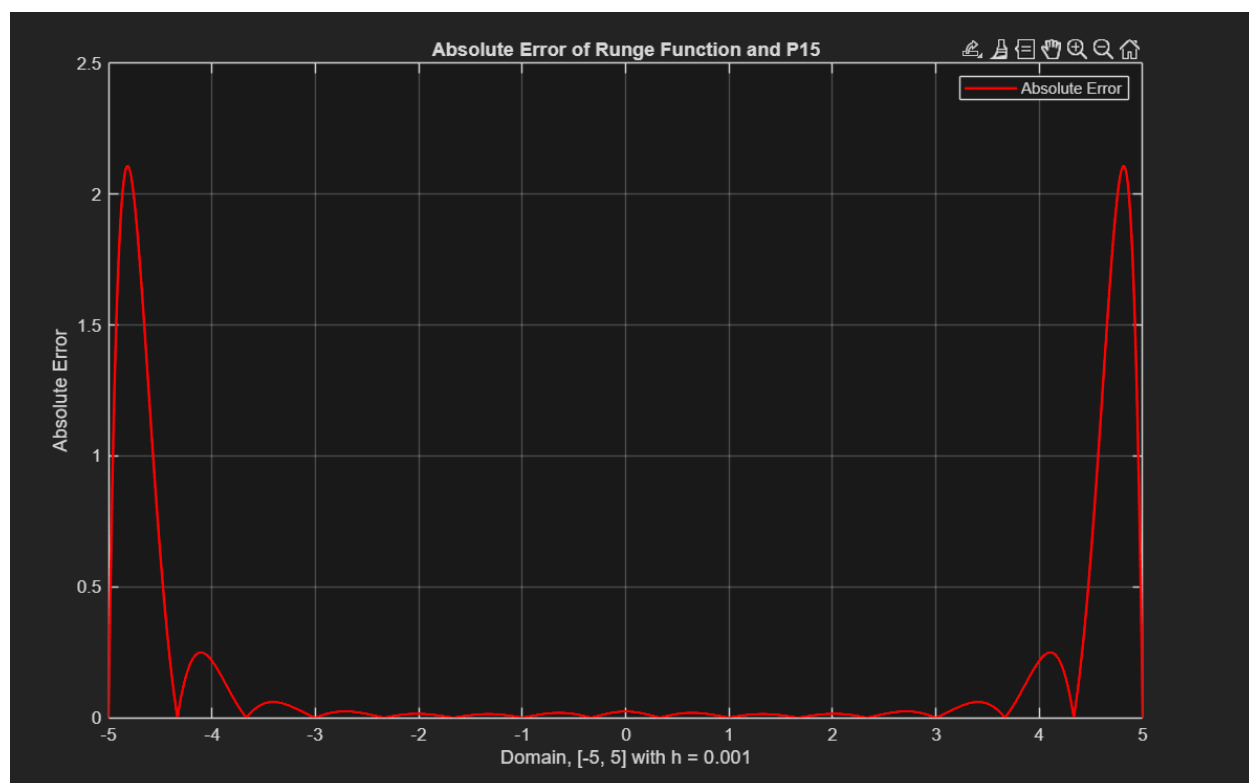
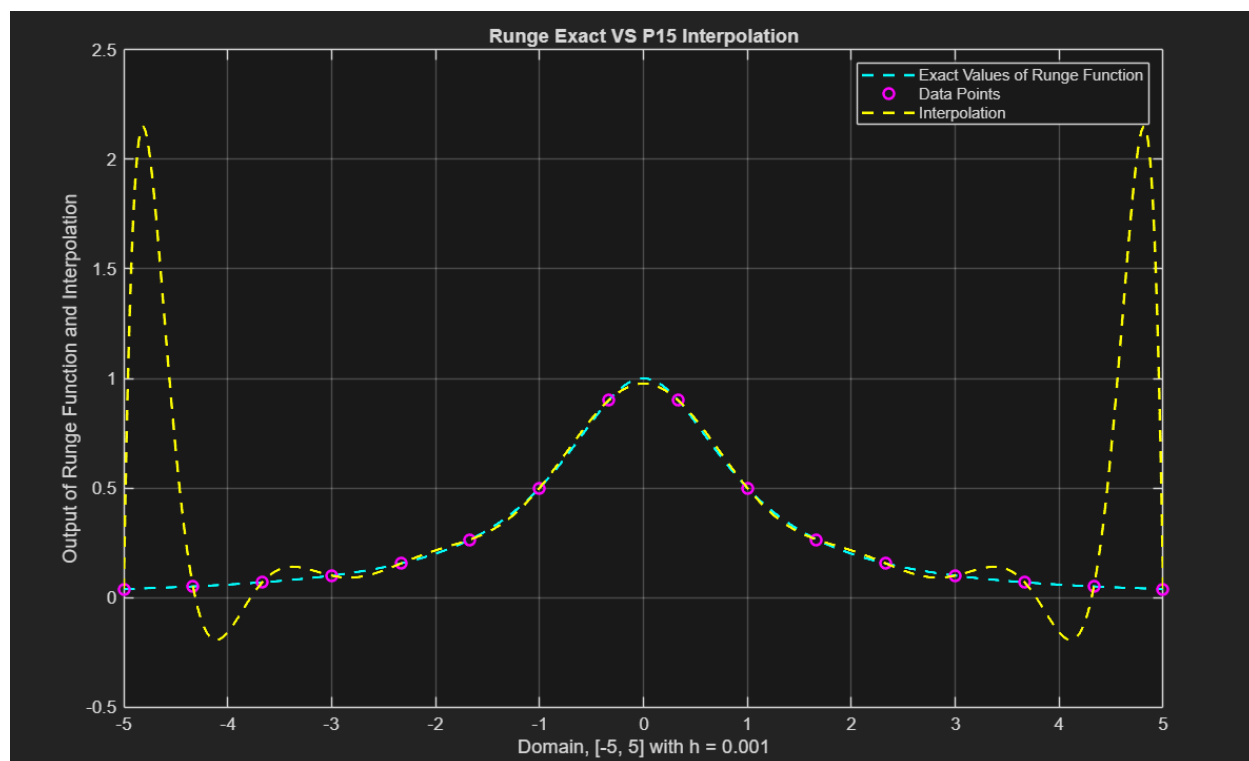
become wildly inaccurate in its approximations to the exact function. We can see this phenomenon even more by investigating the absolute error between the two:



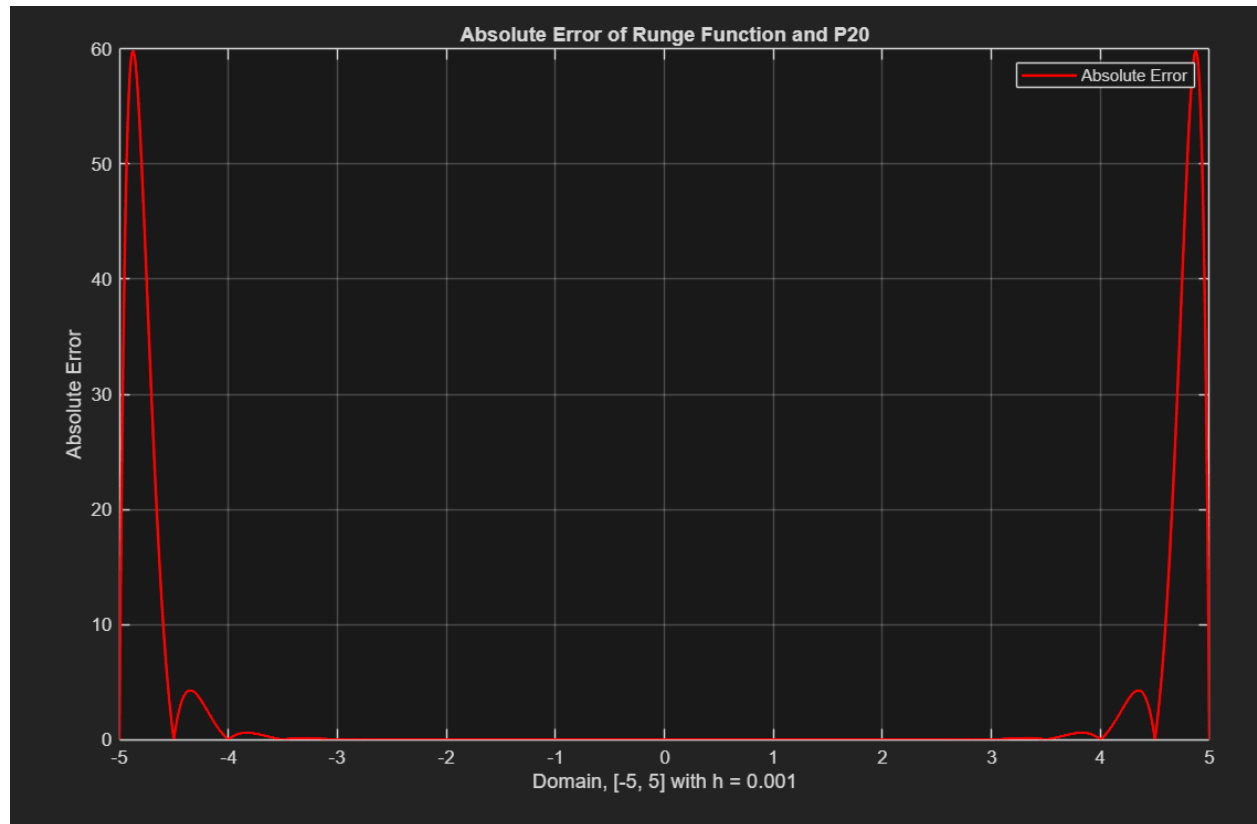
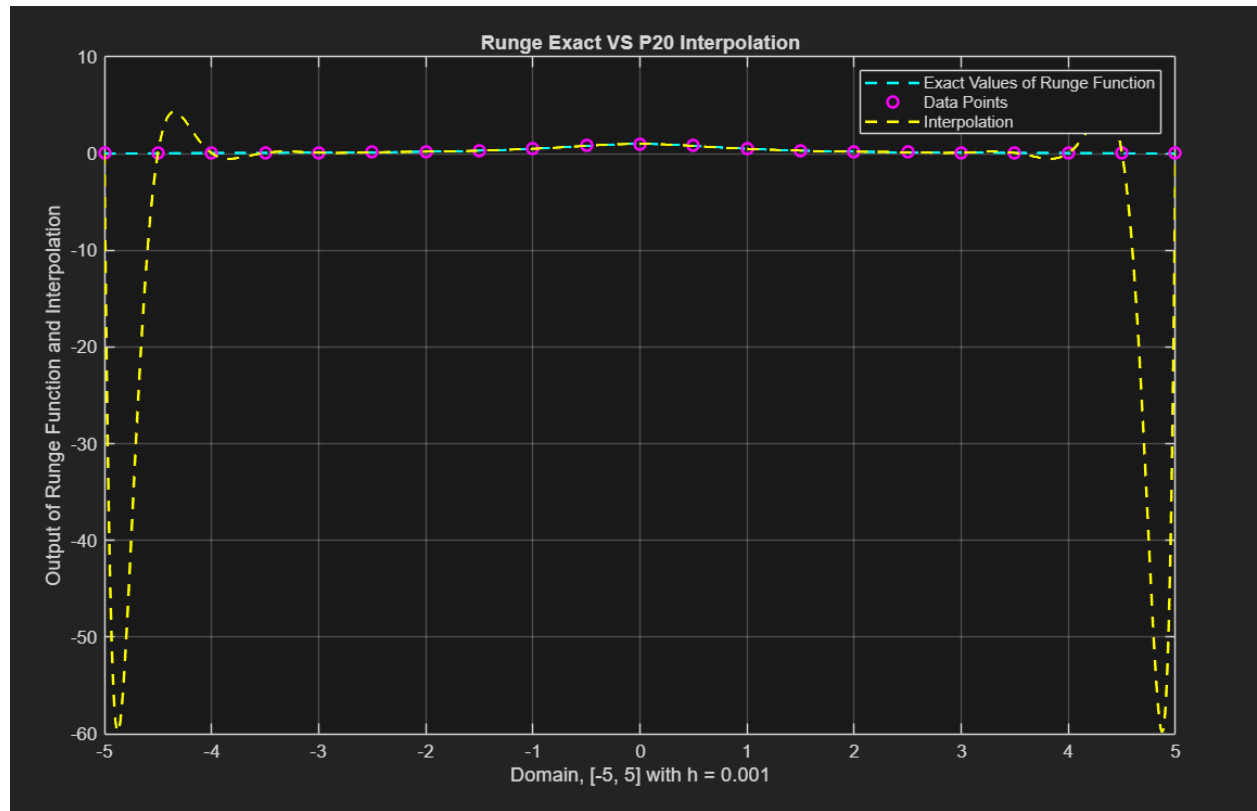
Clearly, as we get further out from the center, the interpolating polynomial becomes a worse approximation.

Simply reading the theorem used in Part 1 (the handworked problem included above) implies a higher degree polynomial should be more accurate. So, to get a higher degree polynomial, let's look at more nodes: $n = 15$, 20 , and 40 .

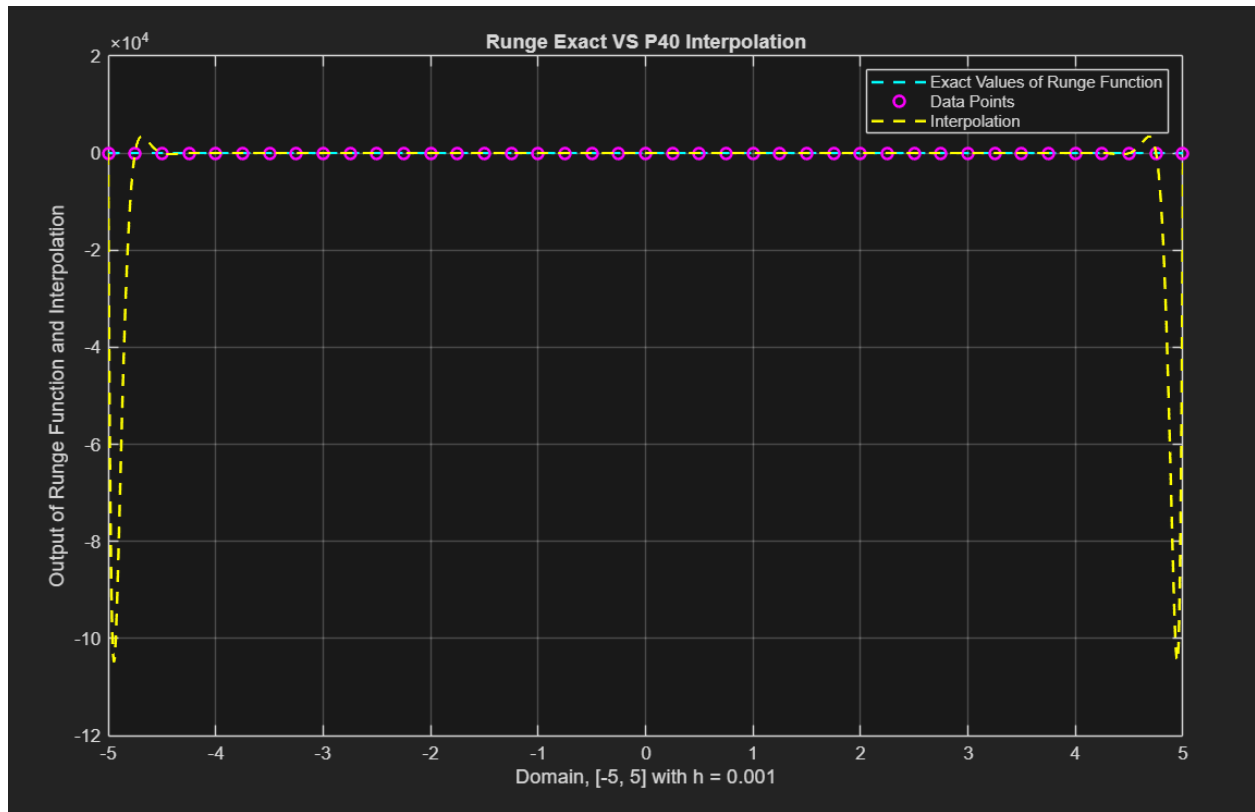
$n = 15$

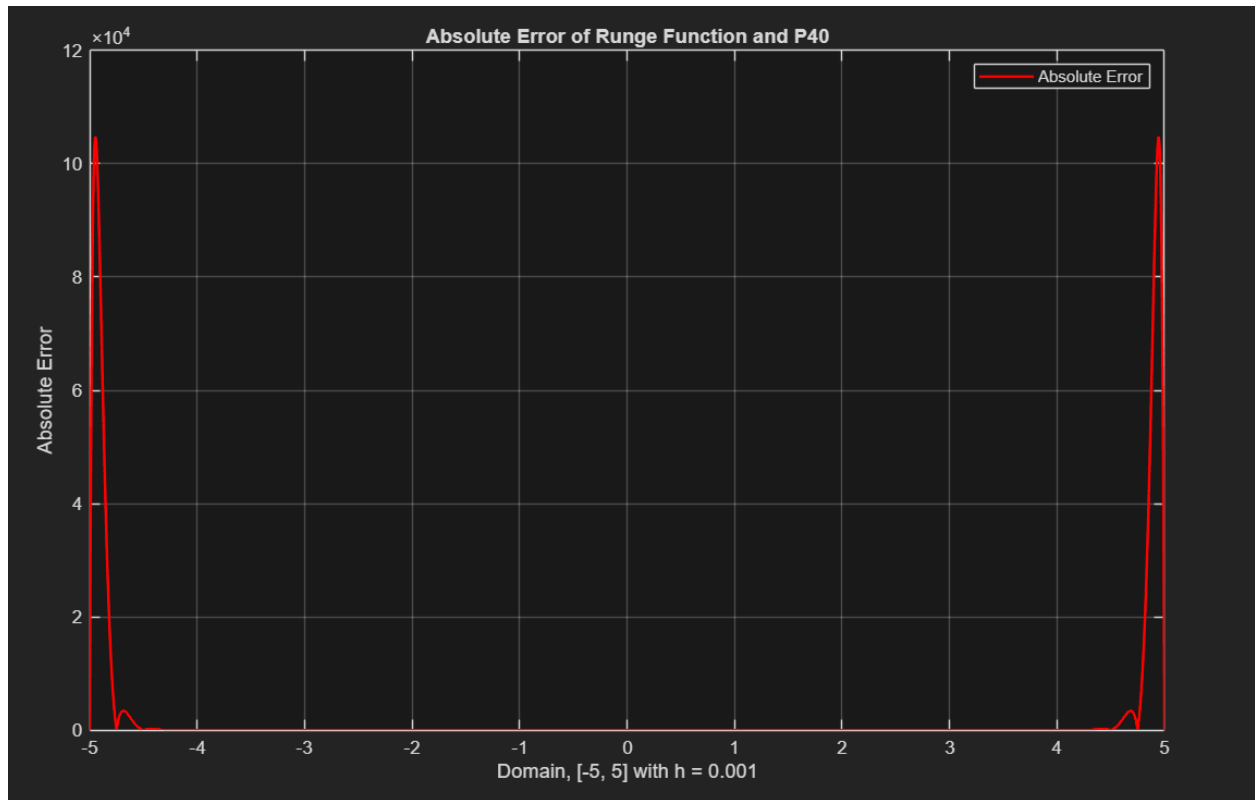


$n = 20$



And finally, $n = 40$





We end up having the complete opposite of the intended effect. Although the middle appears to remain a decent fit, the ends explode in approximation error with the additional degrees. With 41 nodes in the $n = 40$ case, the error at the ends explodes to an order of 10^4 .

Equispaced nodes therefore do not work to approximate the Runge function, and generally potential functions that have derivatives that grow with n . We are completely ignoring the $\max(|f^{(n+1)}(x)|)$ derivative term in the error bound when we assume that more nodes guarantee an improvement. The Runge function is one of the unique functions in which the successive derivative terms yield larger magnitude with every derivative taken. Therefore, we cannot ignore that term in this case, and this phenomenon is evidence of that failure in accounting.

PART B

“Same task as part a) but now consider ChebyShev nodes and label your polynomial as $G_{10}(x)$ ”

To make the task simpler, we will (1) create a function to generate a specified number of Chebyshev nodes given a custom start and endpoint to the interval. For this, we will use the general form equation for generating Chebyshev nodes on a generic interval: $x_k = (a + b) / 2 + ((b - a) / 2) * \cos((2k + 1) * (\pi) / (2n))$ for $k = 0, 1, \dots, n - 1$, where a is the start point of the interval, b is the end point of the interval, and n is the number of nodes to generate. Then, we

will (2) follow through by using *Coef* and *Eval* as done in Part A to get data points and ultimately create an interpolating polynomial.

```
%
=====
(1) GENERATION OF CHEBYSHEV NODES IN chebyshev_nodes.m FILE
%
=====

function [x_GN] = chebyshev_nodes(num_nodes, start_pt, end_pt)
    x_k = @(k) cos((((2 .* k) + 1) .* pi) ./ (2 .* num_nodes));
    x_k_tilde = @(k) ((start_pt + end_pt) ./ 2) + (((end_pt -
start_pt) ./ 2) * x_k(k));

    for k = 0:(num_nodes - 1);
        x_GN(k + 1) = x_k_tilde(k);
    end;
end

%
=====
(2) GENERATION OF THE G10 INTERPOLATION IN script_01.m FILE
%
=====

x_G10 = chebyshev_nodes(n + 1, start_pt, end_pt)
y_G10 = runge_function(x_G10); % evaluate data points
a_coeff = Coef(n, x_G10, y_G10); % get coefficients
% evaluate the interpolating polynomial at a finer mesh
x_G10_Interp = [start_pt:h:end_pt];

for i = 1:length(x_G10_Interp);
    y_G10_Interp(i) = Eval(n, x_G10, a_coeff, x_G10_Interp(i));
end;

figure('Name', 'Interpolation of G10');
plot( ...
    x_exact, y_exact, "--c", ...
    x_G10, y_G10, 'mo', ...
    x_G10_Interp, y_G10_Interp, "--g", ...
    LineWidth=1.5 ...
```

```

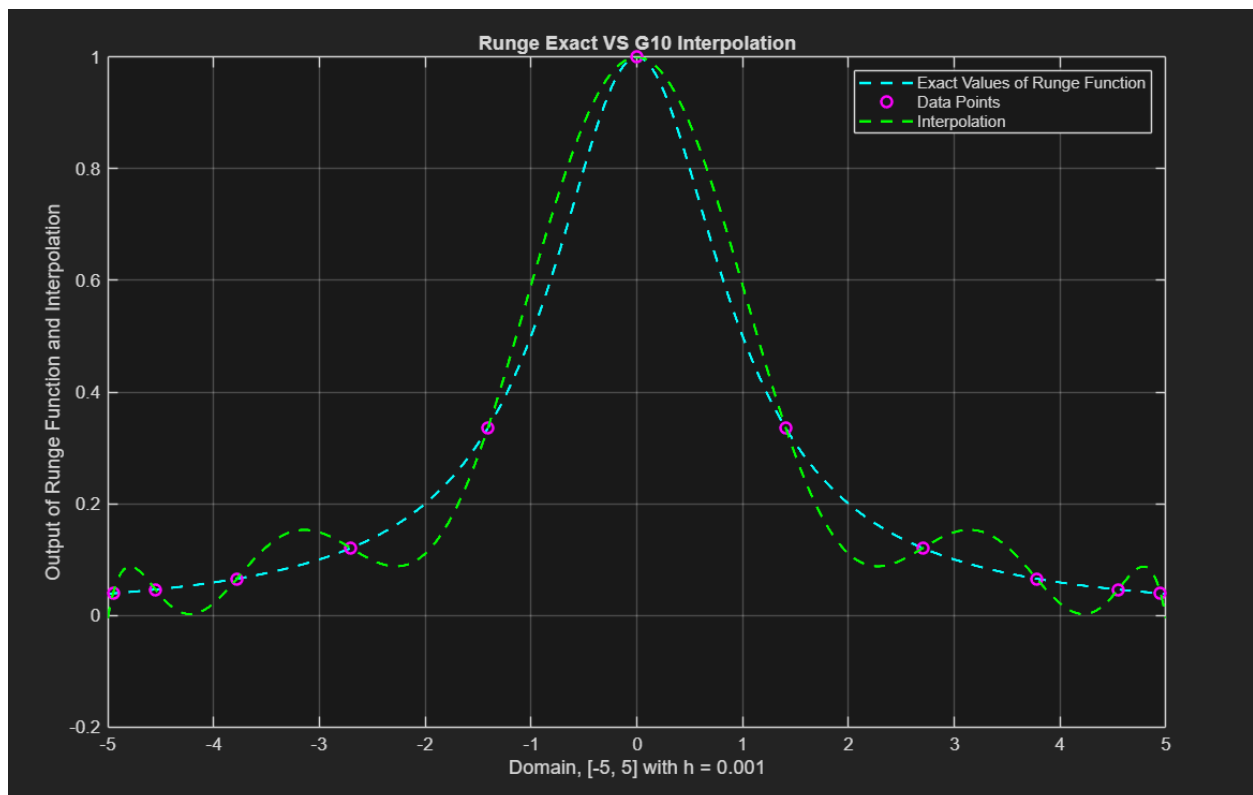
    );
hold on;
legend( ...
    'Exact Values of Runge Function', ...
    "Data Points", ...
    "Interpolation" ...
);
title('Runge Exact VS G10 Interpolation');
xlabel("Domain, [" + start_pt + ", " + end_pt + "] with h = " +
h);
ylabel('Output of Runge Function and Interpolation');
grid on;

%
=====

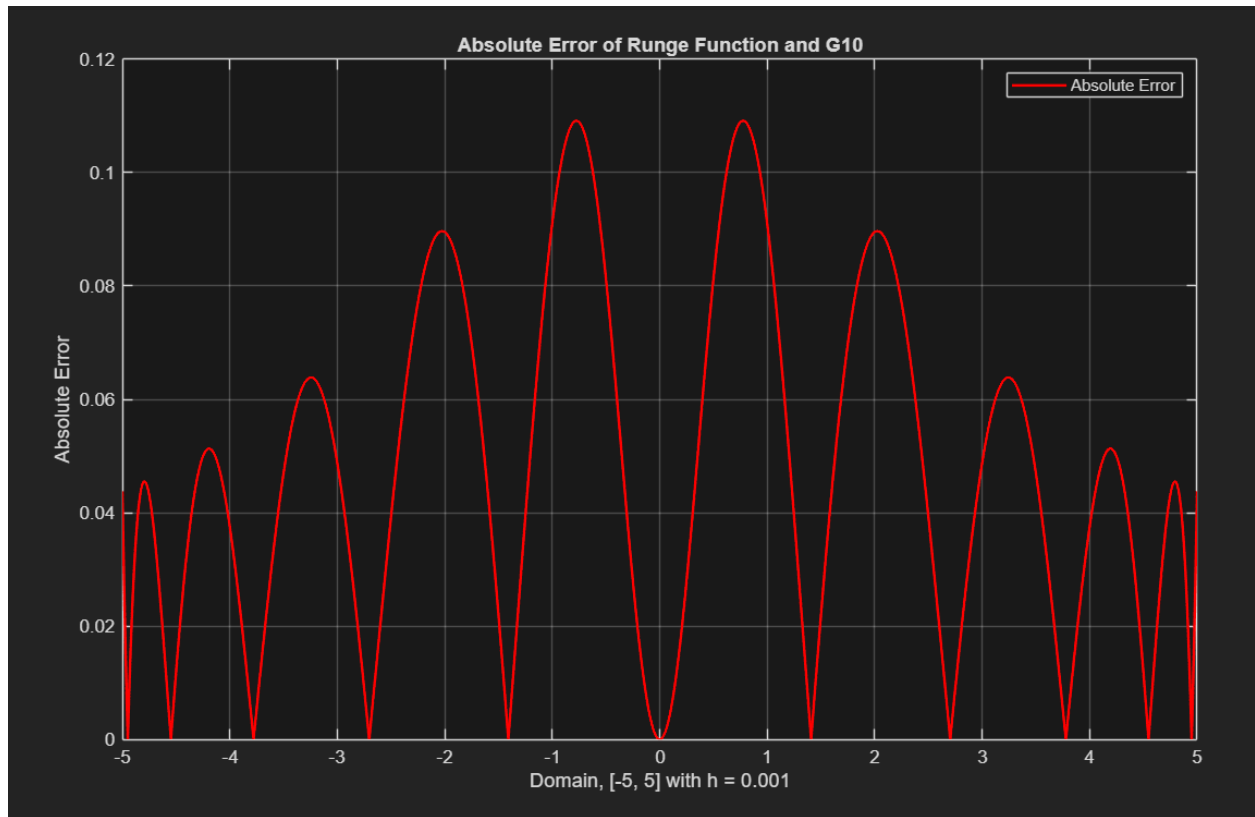
    END CODE FOR COMP PROB 1
%
=====

```

As seen below, we get a much improved approximation from these Chebyshev nodes to the exact Runge values.



We can already see that this approximation lacks the oscillation near the edges of the interval. To have a clearer look, we can also look at the absolute error between G_{10} and exact Runge values.



Although this appears at first glance to be worse than P_{10} , the range of the plot is notable. We will see the comparison more clearly in Part C.

PART C

“Plot $f(x)$, $P_{10}(x)$ and $G_{10}(x)$ vs x in one figure and clearly label your functions. You should use a fine mesh, that is, use more than 11 points so that your plots are smooth.”

We will use the following code to simply plot the two polynomials together against the exact Runge.

```
%
=====
(1) PLOTTING BOTH P10 and G10 IN script_01.m FILE
%
=====
```

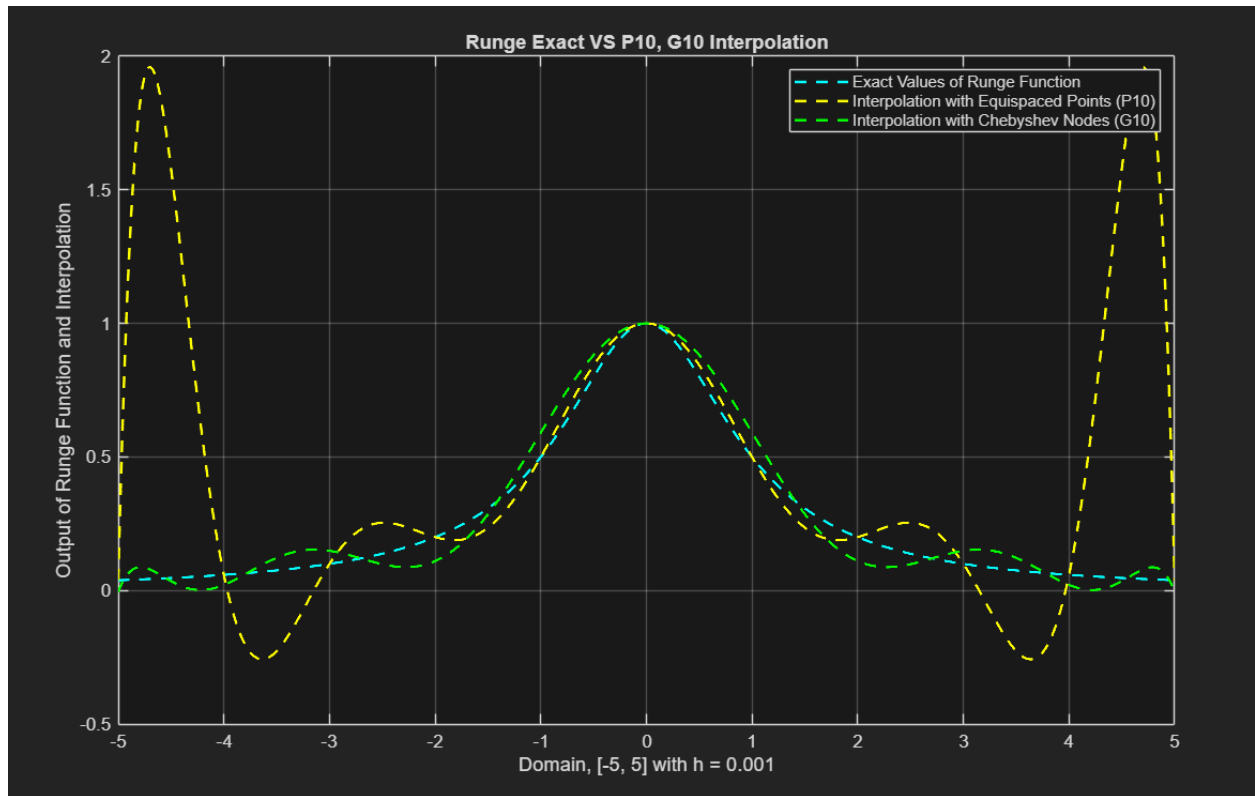
```

figure('Name', 'Interpolation of Both P10 and G10');
plot( ...
    x_exact, y_exact, "--c", ...
    x_P10_Interp, y_P10_Interp, "--y", ...
    x_G10_Interp, y_G10_Interp, "--g", ...
    LineWidth=1.5 ...
);
hold on;
legend( ...
    'Exact Values of Runge Function', ...
    "Interpolation with Equispaced Points (P10)", ...
    "Interpolation with Chebyshev Nodes (G10)" ...
);
title('Runge Exact VS P10, G10 Interpolation');
xlabel("Domain, [" + start_pt + ", " + end_pt + "] with h = " +
h);
ylabel('Output of Runge Function and Interpolation');
grid on;

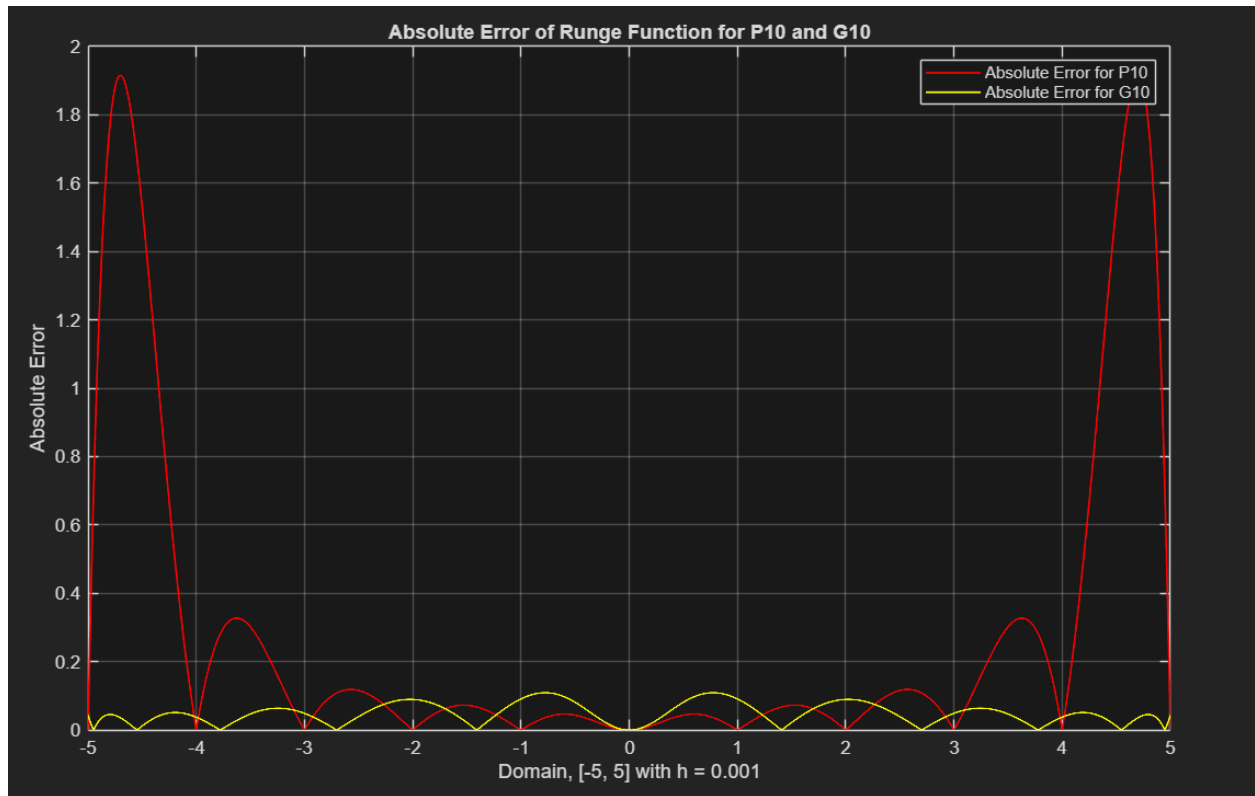
%
=====
END CODE FOR COMP PROB 1
%
=====

```

When these are plotted together, we can see a clear difference.



The Chebyshev nodes provide a much tighter polynomial interpolation than equispaced nodes for the Runge function. We can also see this clearly in the error plotting of the two interpolations together.



The $G10$ interpolation stays quite tight to the exact values (even at the edges), while $P10$ varies wildly as it moves away from center. These data points are likely more useful in approximating this function based on this observation. Chebyshev nodes are far more concentrated at the ends of the interval, which appears to have a definitive impact on improving the wild oscillations apparent in equispaced nodes.

PART D

“To test things out, plot $G20(x)$, $G40(x)$ and $G60(x)$ and discuss your findings. Provide an educated guess of your results. Fix your window axis([-5 5 -0.5 2])”

As we saw in Part A, a higher degree polynomial with equidistant points was a failure in terms of accuracy on the entire domain. So, these plots should show if the same remains true by using Chebyshev spacing.

The figure plotted in the submitted code (as seen below) will contain all 3 plots, along with the exact Runge, but the plots shown will show-case them all separately.

So, first, we will (1) write code in a similar fashion to the first few parts to generate plots of $G20$, $G40$, and $G60$.

```

%
=====

(1) GENERATION OF CHEBYSHEV NODES IN chebyshev_nodes.m FILE
%
=====

% n = 20
n = 20;
x_G20 = chebyshev_nodes(n + 1, start_pt, end_pt);
y_G20 = runge_function(x_G20); % evaluate data points
a_coeff = Coef(n, x_G20, y_G20); % get coefficients
% evaluate the interpolating polynomial at a finer mesh
x_G20_Interp = [start_pt:h:end_pt];
for i = 1:length(x_G20_Interp);
    y_G20_Interp(i) = Eval(n, x_G20, a_coeff, x_G20_Interp(i));
end;

% n = 40
n = 40;
x_G40 = chebyshev_nodes(n + 1, start_pt, end_pt);
y_G40 = runge_function(x_G40); % evaluate data points
a_coeff = Coef(n, x_G40, y_G40); % get coefficients
% evaluate the interpolating polynomial at a finer mesh
x_G40_Interp = [start_pt:h:end_pt];
for i = 1:length(x_G40_Interp);
    y_G40_Interp(i) = Eval(n, x_G40, a_coeff, x_G40_Interp(i));
end;

% n = 60
n = 60;
x_G60 = chebyshev_nodes(n + 1, start_pt, end_pt);
y_G60 = runge_function(x_G60); % evaluate data points
a_coeff = Coef(n, x_G60, y_G60); % get coefficients
% evaluate the interpolating polynomial at a finer mesh
x_G60_Interp = [start_pt:h:end_pt];
for i = 1:length(x_G60_Interp);
    y_G60_Interp(i) = Eval(n, x_G60, a_coeff, x_G60_Interp(i));
end;

% plot all together

```

```

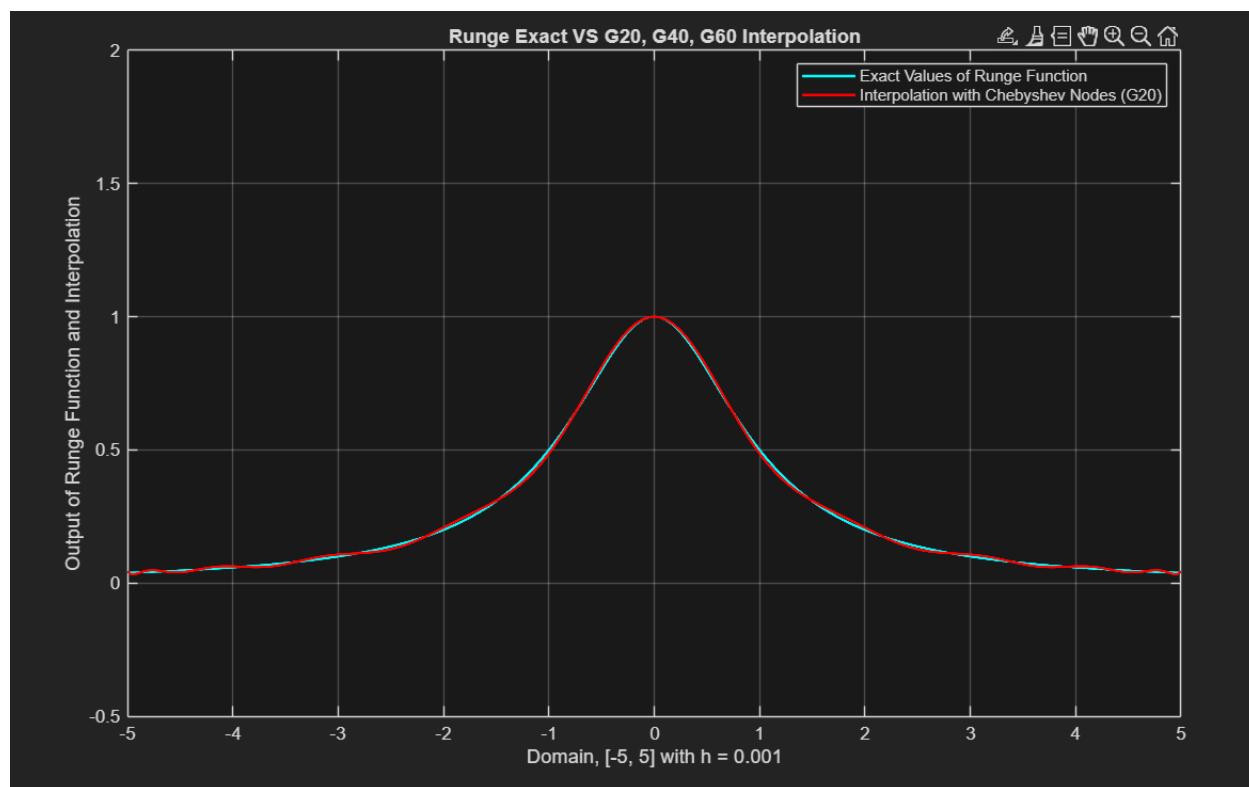
figure('Name', 'Interpolation of G20, G40, G60');
plot( ...
    x_exact, y_exact, "-c", ...
    x_G20_Interp, y_G20_Interp, "--r", ...
    x_G40_Interp, y_G40_Interp, "--y", ...
    x_G60_Interp, y_G60_Interp, "--g", ...
    LineWidth=1.5 ...
);
hold on;
legend( ...
    'Exact Values of Runge Function', ...
    "Interpolation with Chebyshev Nodes (G20)", ...
    "Interpolation with Chebyshev Nodes (G40)", ...
    "Interpolation with Chebyshev Nodes (G60)" ...
);
title('Runge Exact VS G20, G40, G60 Interpolation');
xlabel("Domain, [" + start_pt + ", " + end_pt + "] with h = " +
h);
xlim([-5, 5]); % limit specified
ylabel('Output of Runge Function and Interpolation');
ylim([-0.5, 2]); % limit specified
grid on;

%
=====
    END CODE FOR COMP PROB 1
%
=====

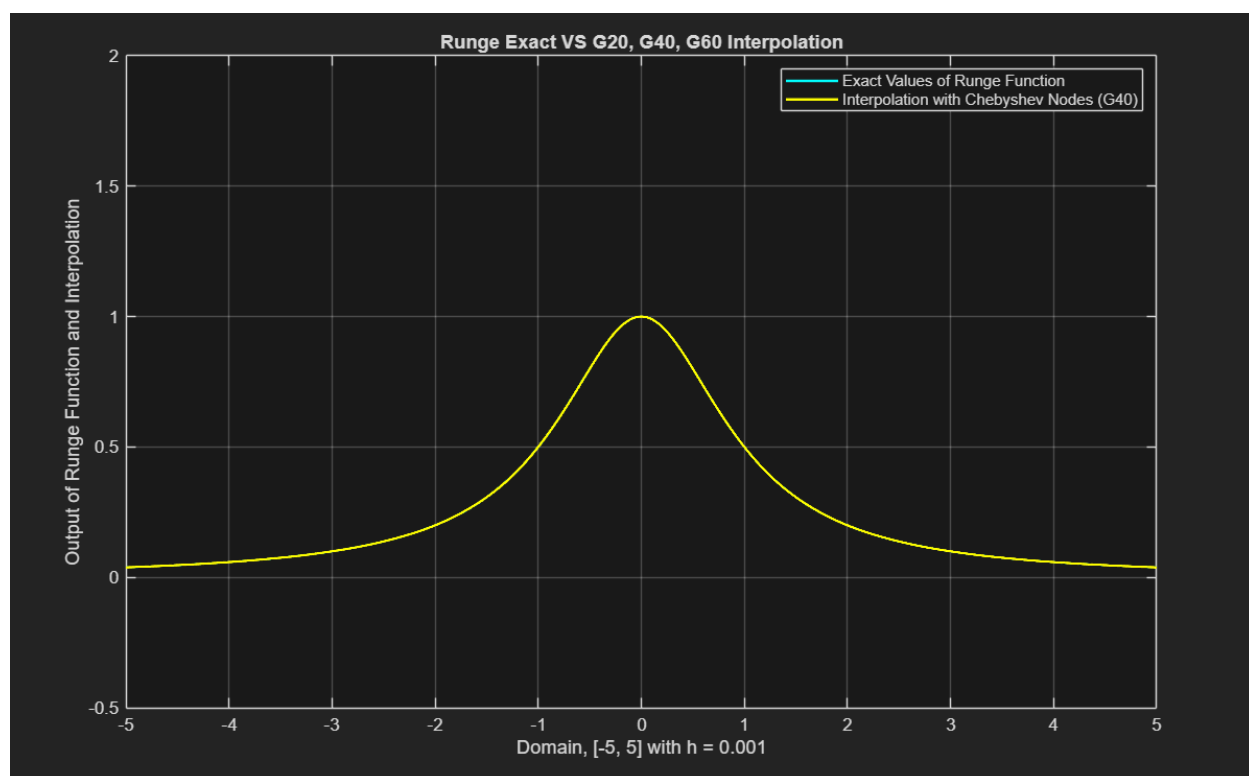
```

And now, we will see each of these approximations in isolation.

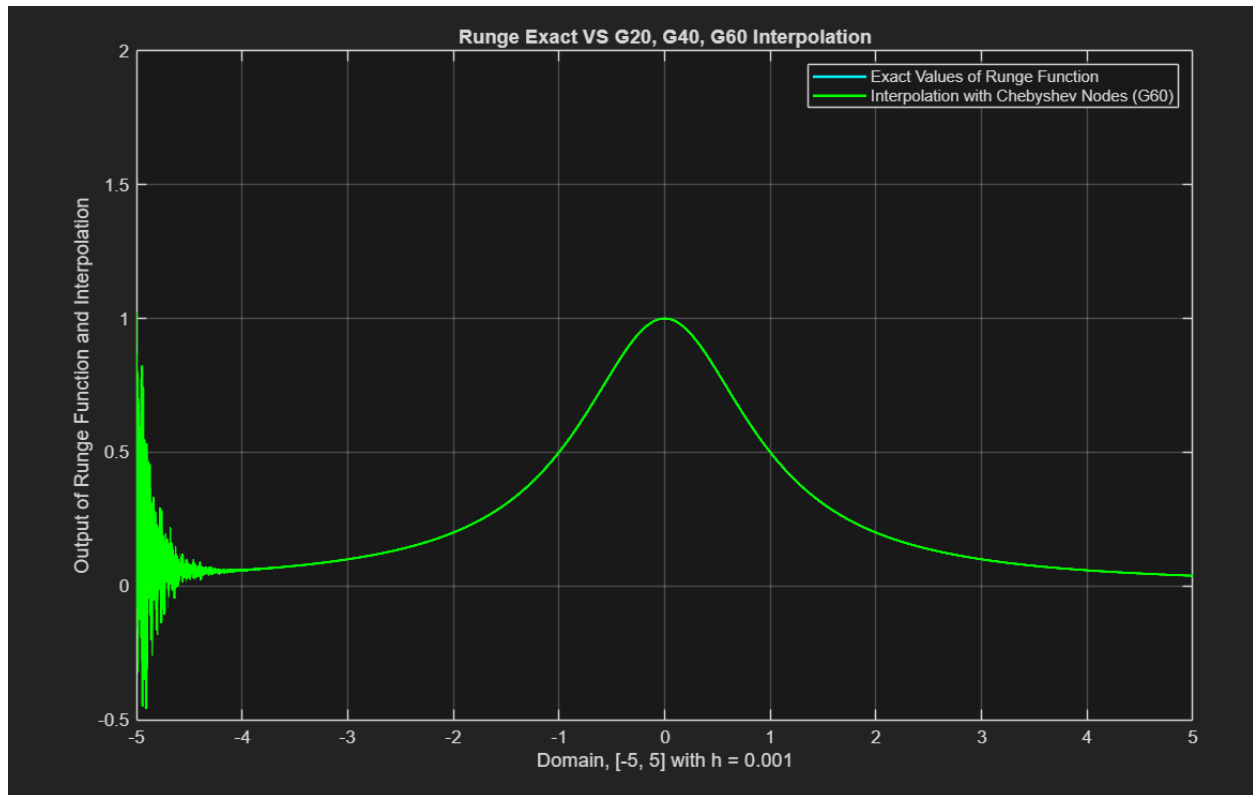
$n = 20$



$n = 40$

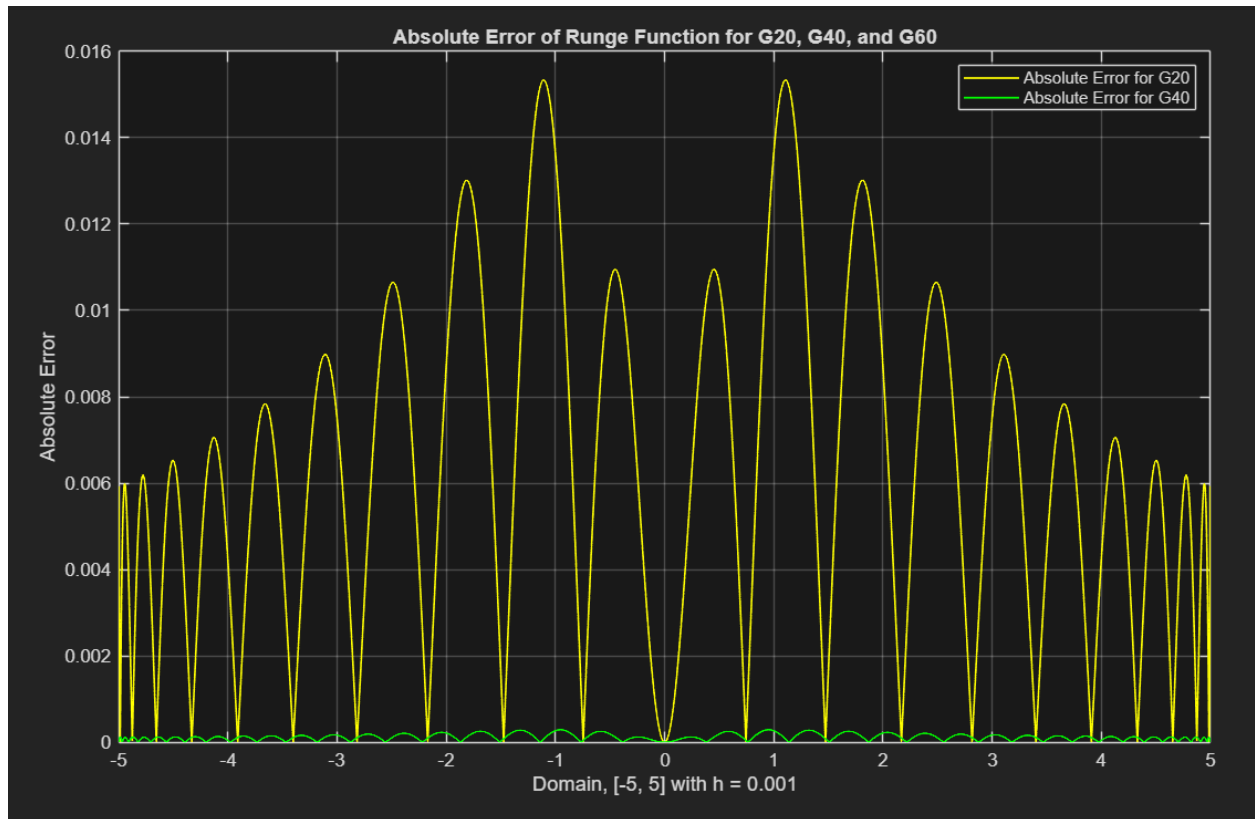


And lastly, $n = 60$

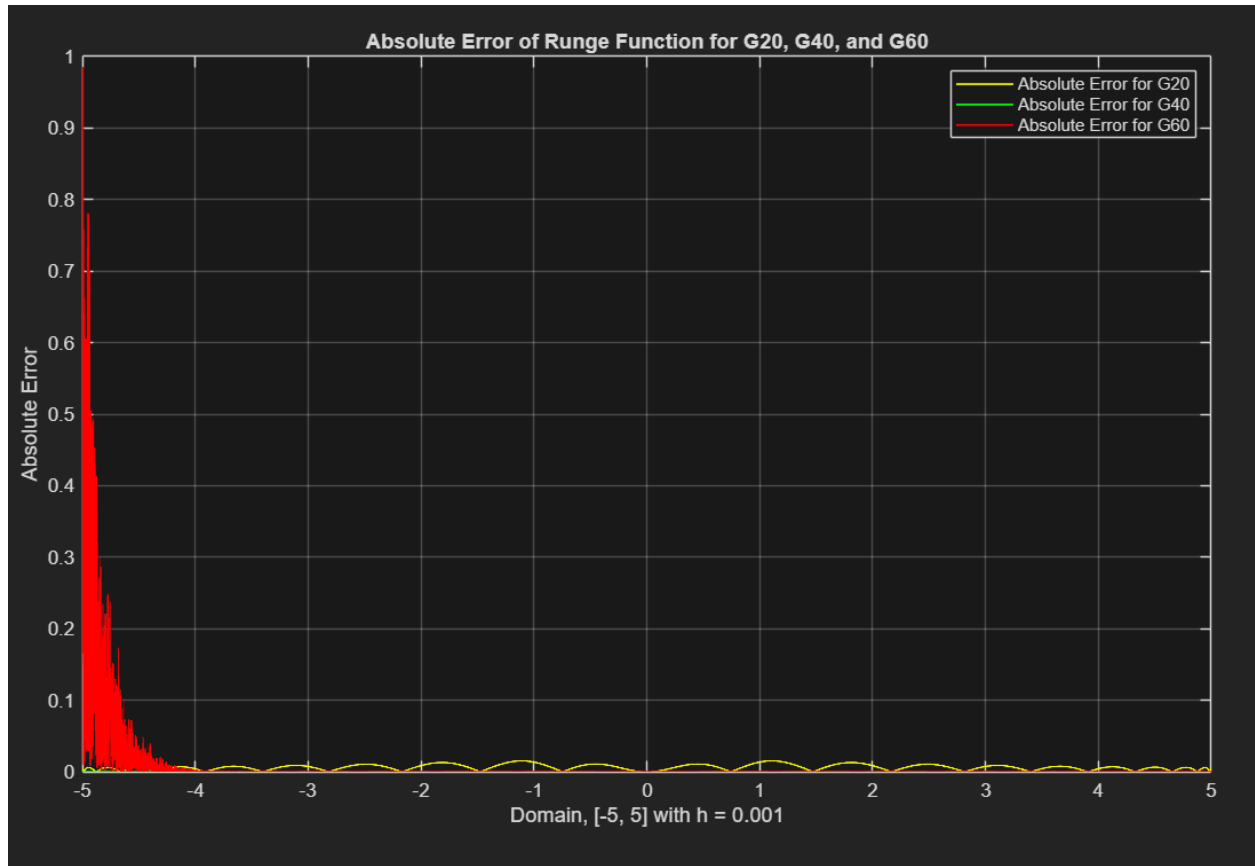


We see that the higher degrees appear to be increasing accuracy—right up until $n = 60$, where we see some strange oscillations near the negative extreme of the interval. We can see the comparisons clearly as well while looking at error plottings.

First, we can see below the stunning decrease in an already small error from $G20$ to $G40$.



Further, the G_{20} approximation is an order of magnitude improvement on G_{10} , which had the highest error magnitude of slightly above 0.1 as seen in the plotting for Part B. However, once adding in G_{60} , the strange oscillation is apparent, defeating our expectations of uniform increase in accuracy with 61 nodes.



The oddity worth noting here is not only that, past a certain point, accuracy suffers, but additionally it is only the one extreme that suffers on this domain, despite the function and approximations being symmetric.

Clearly Chebyshev nodes are spaced in such a way to minimize the oscillations that could occur with Runge's function, being densely concentrated near the ends. And it does appear to assist in accuracy improvements with higher n , and has a far higher error tolerance with higher degree interpolation. However, there is likely a line at which, when crossed, the Chebyshev nodes still cannot minimize the oscillation error enough to provide high accuracy. The derivative is still a part of the upper bound error, and cannot be completely ignored, regardless of minimizing the other terms within the error bound. So, within reason, Chebyshev can mitigate this error more, but only to a certain point.

CONCLUSION

In conclusion, although we can use a polynomial of high degree to approximate function, it is still important to consider all terms that affect the error of interpolation. In typical cases, this can

be an incredibly useful tool to approximate difficult functions with a far nicer function. Though, there are unique functions, like the Runge function, that will prove uniquely difficult to accomplish this accurately with. The Runge phenomenon highlights the importance of considering the mathematics of error and using methods wisely in specific situations. Yet again, it is our responsibility to consider these potential pitfalls and use methods wisely.